

FreeRADIUS

FreeRADIUS (<https://freeradius.org>) is one of the top open source **RADIUS** servers. FreeRADIUS can be used as an Authentication Server in 802.1X and therefore for WPA/WPA2/WPA3 Enterprise setup. More information about IEEE 802.1X and WPA Enterprise you can find in [802.1X Port-Based Authentication HOWTO](https://fdp.org/HOWTO/html_single/8021X-HOWTO/) (https://fdp.org/HOWTO/html_single/8021X-HOWTO/). FreeRADIUS can be set up rather easily with the default configuration and minimal changes.

This guide will cover FreeRADIUS 3 (OpenWrt package version `freeradius3_3_0_21-1`) installation on OpenWrt 19.07.4 and configuration for WPA2 Enterprise. The described configuration was tested on Banana Pi M1 as a FreeRADIUS server and ZyXEL Keenetic as a Wireless Access Point. FreeRADIUS should also work on devices with significantly less flash and RAM, for example TP-Link TD-W8980 V1 running OpenWrt v18.06.1.

Notes:

- It is possible (<https://forum.openwrt.org/t/using-wpad-as-radius-server/54625/4>) to configure WPA/WPA2/WPA3 Enterprise only using `hostapd` , but at the moment this is not feasible using standard OpenWrt methods.
- If you are looking for **FreeRADIUS 2** you may find this blog post (<http://www.blog.10deam.com/2015/01/08/install-freeradius2-on-a-openwrt-router-for-eap-authentication/>) helpful.

Installation

To install FreeRADIUS 3 on OpenWrt, which can run its default configuration, simply run:

```
# opkg update && opkg install freeradius3-default
```

You can also install the `freeradius3-utils` package for basic testing and monitoring of FreeRADIUS:

```
# opkg install freeradius3-utils
```

Of course, you can install the main `freeradius3` package and only those `freeradius3-mod*` packages that are used by your modified configuration files.

Older OpenWrt releases

If meta-package `freeradius3-default` is not present in your OpenWrt release, install the packages one by one:

```
opkg update
opkg install freeradius3 freeradius3-common freeradius3-democerts freeradius3-mod-always freeradius3-mod-attr-filter freeradius3-mod-chap freeradius3-mod-detail freeradius3-mod-digest freeradius3-mod-eap freeradius3-mod-eap-gtc freeradius3-mod-eap-leap freeradius3-mod-eap-md5 freeradius3-mod-eap-mschapv2 freeradius3-mod-eap-peap freeradius3-mod-eap-tls freeradius3-mod-eap-ttls freeradius3-mod-exec freeradius3-mod-expiration freeradius3-mod-expr freeradius3-mod-files freeradius3-mod-ldap freeradius3-mod-logintime freeradius3-mod-mschap freeradius3-mod-pap freeradius3-mod-passwd freeradius3-mod-preprocess freeradius3-mod-radutmp freeradius3-mod-realm freeradius3-mod-unix freeradius3-utils
```

Or you can also just install all of the above with another command but this is not recommended:

```
eval $(opkg find freeradius3* | sed 's/ - .*//' | sed 's/^/opkg install /')
```

Note:

- In rare occasions, while installing the above packages, you may encounter an error saying that the same file is being provided by two packages. If this happens, delete the offending file and look for the yet-not-installed package due to the error and install it again.

Configuration

According (<http://deployingradius.com/documents/configuration/setup.html>) to the authors of FreeRADIUS, the default configuration is designed to work everywhere, and to perform nearly every authentication method. They recommend using a revision control system such as git or Mercurial on the configuration files and testing the configuration after each change. For a small installation, using a revision control system is an overkill. However, it is helpful to comment on the changes you made in the configuration files so that you can then easily find *what* you changed and *why*.

Please make sure you have terminal access to your OpenWrt device via `SSH()` or serial connection, because this will be necessary for this guide to work. You can also use WinSCP for Windows to edit files easily.

Terms

This and the next section are mainly based on the PDF guides from here (<https://networkradius.com/technology/freeradius/>).

A glossary of some of the terms used below.

- **AAA** - Authentication, Authorization, and Accounting - a security architecture for distributed systems that provides control over user access to services and resources and tracks user activities.
- **RADIUS** - Remote Authentication Dial In User Service - a network protocol for remote user authentication and accounting.
- **FreeRADIUS** - a modular, high performance, open source variant of RADIUS server. Can be used as an Authentication Server.
- **Authentication Server** - processes authentication requests from the NAS. Optionally requests user and configuration information from the database or directory. May return configuration parameters to the NAS, for example `Access-Accept` or `Access-Reject` . Receives accounting information from the NAS. The server has no way of knowing if

the NAS has received its response, or if the NAS is obeying the instructions in that response.

- **Supplicant** - a user's device or its component that is responsible for responding to Authenticator data that will establish its credentials.
- **NAS** - Network Access Server, also **Authenticator** - provides or blocks access to the network for the user/device (Supplicant). Typically the Authenticator is a part of wireless access points such as the Linksys WRT54G, network switches and dial-up equipment. NAS acts as a client to a RADIUS server. EAPOL is used between the Supplicant and the Authenticator; and, between the Authenticator and the Authentication Server, RADIUS is used. During the authentication process, the Authenticator just relays all messages between the Supplicant and the Authentication Server.
- **PAP** - Password Authentication Protocol - an authentication protocol that uses an unencrypted ASCII() password.
- **CHAP** - Challenge-Handshake Authentication Protocol - a protocol that authenticates network users. Client creates a random string, called the challenge, and performs a weak¹⁾ MD5 hash to combine the challenge with the password.
- **MS-CHAPv1** and **MS-CHAPv2** - the Microsoft (MS) CHAP mechanisms are similar to the CHAP method. Has known weaknesses²⁾.
- **EAP** - Extensible Authentication Protocol - an authentication framework. Variants like EAP-TTLS and EAP-TLS() add the Transport Layer Security protocol.
- **EAPOL** - EAP encapsulation over LAN().
- **EAP-MD5** - CHAP wrapped in EAP.
- **EAP-MSCHAPv2** - MS-CHAPv2 wrapped in EAP.
- **EAP-TLS()** - the Transport Layer Security (TLS()) authentication method provides a TLS() tunnel between the Supplicant and RADIUS server. It requires server and client certificates to operate.
- **EAP-TTLS** - the Tunneled TLS() (TTLS) method is similar to EAP-TLS(). However, in this case, client certificates are not necessary. Instead, the TLS() tunnel is used to transport additional authentication data such as PAP, CHAP, MS-CHAP or other.
- **PEAP** - Protected Extensible Authentication Protocol - a Microsoft created protocol that encapsulates EAP in an encrypted and authenticated TLS() tunnel. It is broadly similar to EAP-TTLS, but the difference is that the authentication method carried inside of the TLS() tunnel in PEAP is identical to MS-CHAPv2.

Key FreeRADIUS configuration files

In OpenWrt, FreeRADIUS stores its configuration in the `/etc/freeradius3` directory. In the official FreeRADIUS documentation, the configuration directory is named **raddb**.

The configuration files themselves contain enormous amounts of documentation. Each example has comments describing what it does, when it should be used, and how to configure it. Please note that due to the OpenWrt packaging policy, FreeRADIUS distribution in OpenWrt does not include many files, such as readmes, scripts and examples. You can find them by installing FreeRADIUS on a full-sized Unix-like distribution like Ubuntu or Fedora, or in the FreeRADIUS source code. This full-featured FreeRADIUS installation can also be useful for tests and certificates generation which are described below.

- **radiusd.conf** - the main configuration file. It includes references to all of the other configuration files.
- **clients.conf** - defines information necessary to communicate with the RADIUS clients (NAS), including IP() addresses and shared secrets.
- **sites-enabled/default** - this is the default virtual server. This file configures the handling of authentication and accounting requests. It contains a configuration designed to work with the largest number of authentication protocols.
- **sites-enabled/inner-tunnel** - this virtual server handles authentication methods that are carried inside of a TLS() tunnel, as part of PEAP or EAP-TTLS authentication.
- **mods-config/files/authorize** - the traditional RADIUS configuration file which contains usernames and passwords. In previous FreeRADIUS releases this file was `raddb/users`.

- **certs/** - this sub-directory stores `TLS()` certificates.
- **sites-available/** - a directory to store virtual servers that administrators can enable. Each virtual server encapsulates one logical set of functionality.
- **sites-enabled/** - a directory to store active virtual servers. These are usually symbolic links to files in the `sites-available/` directory.
- **mods-available/** - a directory to store module configurations files that network administrators can enable on an optional basis. Each module is configured in a different file in the directory.
- **mods-enabled/** - a directory to store active module configuration files. These are usually symbolic links to files in the `mods-available/` directory.
- **mods-config/** - module-specific configuration files.

Note:

- You can view contents of a configuration file without comments using command

```
grep -v -e '^\\s*#' -e '^$' /etc/freeradius3/radiusd.conf | less
```

Set up FreeRADIUS for initial testing

Let's check that FreeRADIUS can run properly in debugging mode and authenticate a test user via the simplest PAP. In debugging mode, the server prints its configuration to the current terminal window, and then details of every request. This is especially useful when you are trying to understand how the server works. The chapter is based on [this](http://deployingradius.com/documents/configuration/pap.html) (http://deployingradius.com/documents/configuration/pap.html) HOWTO.

If you just installed FreeRADIUS, it will start running automatically. It's better to stop it before changing config for the first time:

1. Find out if FreeRADIUS is running:

```
# ps | grep [r]adiusd
```

2. If there is some output, stop FreeRADIUS:

```
# /etc/init.d/radiusd stop
```

The default configuration contains a user `bob` with password `hello` , but it needs to be uncommented before it can be used. Edit the `authorize` file in `/etc/freeradius3/mods-config/files/` and uncomment the lines below. If you cannot find the lines, add them to the top of the file.

```
bob      Cleartext-Password := "hello"
         Reply-Message := "Hello, %{User-Name}"
```

Please make sure, you insert an indent after the name of the person with a 'Tab' (space should also work). The tab-indented `Reply-Message` option is not mandatory and can be omitted.

If you have installed the `freeradius3-utils` package containing the `radtest` utility along with `freeradius3` , then the preparation for the test is complete.

If, for some reason, this package and the utility are not installed, you can run a test from a machine accessible over the network with FreeRADIUS and `radtest` installed. Additionally, you need to define this machine as a client (NAS) to your RADIUS server. With the default `clients.conf` configuration, the server is ready to accept connections only from NAS which has IP() `127.0.0.1` (loopback) and `secret = testing123` . Edit the `/etc/freeradius3/clients.conf` file to add your machine and add the following lines to the end of the file:

```
# my laptop
client laptop {
    ipaddr = 192.168.1.101    # change it according to the IP address of your test machine
    secret = testing123      # you will need it for testing purposes
}
```

Starting FreeRADIUS in debugging mode

Once you have completed the config, it's time to start FreeRADIUS. In order to see whether the server is working properly, you need to start it in debugging mode. Open a terminal session via `SSH()` or serial connection to the OpenWrt device and start `radiusd` in debugging mode, as user `root`:

```
# radiusd -X
```

After executing the command, if the server is installed and configured correctly, the screen will show a large amount of text that ends with the message **Ready to process requests** . If the server does not start up correctly, the debug output will tell you why. Please make sure to correct any errors before proceeding. You may stop `radiusd` by pressing `Ctrl-C` .

In older OpenWrt releases and `freeradius3` packages, the command may be:

```
LD_LIBRARY_PATH=/usr/lib/freeradius3 radiusd -X
```

Note:

- Setting `LD_LIBRARY_PATH` is important as documented on [GitHub \(https://github.com/openwrt/packages/issues/7667\)](https://github.com/openwrt/packages/issues/7667) because `radiusd` normally looks for it's modules in `/etc/lib` directory but in OpenWrt these are located in `/etc/lib/freeradius3` and this is why you first need to set the variable.

PAP testing

Now your FreeRADIUS server is running and it's time to send a test request and view the response. Start another terminal session to your OpenWrt device (or use your machine with `radtest` , if you set up it previously) and run the following command. Do not forget to change `localhost` to the actual `IP..()` address of your OpenWrt device, if you test from a separate machine.

```
radtest bob hello localhost 0 testing123
# bob is username.
# hello is password.
# localhost is the DNS name or IP address of your OpenWrt device where the radiusd is running.
# 0 is the NAS-port-number. It really doesn't matter what you put here.
# testing123 is the secret for client.
```

In some configurations, it may be necessary to run `radtest` with all parameters specified:

```
radtest -t pap bob hello 127.0.0.1 0 testing123 0 localhost
```

If all goes well, you should see the server returning an `Access-Accept` message, and the window with `radtest` should print text similar to the following:

```
Sent Access-Request Id 141 from 0.0.0.0:55063 to 127.0.0.1:1812 length 73
  User-Name = "bob"
  User-Password = "hello"
  NAS-IP-Address = 127.0.0.1
  NAS-Port = 0
  Message-Authenticator = 0x00
  Cleartext-Password = "hello"
Received Access-Accept Id 141 from 127.0.0.1:1812 to 127.0.0.1:55063 length 20
```

At the same time, you will see text flowing through in the terminal where `radiusd -X` is running. At the end `radiusd` will show something like `Sent Access-Accept Id 141 from 127.0.0.1:1812 to 127.0.0.1:55063 length 0`

If you get any other output, it means something is wrong and you have not configured your FreeRADIUS server properly. Therefore, you need to check the output for errors and fix them. In most cases, the debugging data tells you which file is causing the problem and on which line.

PEAP/MS-CHAPv2 testing

The `radtest` utility does not know how to work with the secure protocols `EAP-TLS..()`, `PEAP`, and `EAP-TTLS`. For such testing, you can ~~(http://deployingradius.com/scripts/eapol_test/)~~ use the `eapol_test` program from `wpa_supplicant` (https://w1.fi/wpa_supplicant/). `eapol_test` utility integrates IEEE 802.1X Authenticator (normally, an access point) and IEEE 802.1X Supplicant (normally, a wireless client) together to generate a single program that can be used to test EAP methods

without having to setup an access point and a wireless client. You can find `eapol_test` in OpenWrt distribution and in some full-featured Unix-like distributions, for example, Fedora. Alternatively, you can compile it from the `wpa_supplicant` sources using the following instructions:

- `wpa_supplicant` documentation (https://w1.fi/wpa_supplicant/devel/testing_tools.html#eapol_test)
- FreeRADIUS documentation (http://deployingradius.com/scripts/eapol_test/)

Note:

- Further instructions assume that `eapol_test` will be installed and used on the same OpenWrt device that is running FreeRADIUS. If it is installed on a separate machine, correct the `IP()` addresses and define this machine as a client (NAS) to your FreeRADIUS server as described above.

Install `eapol_test` in OpenWrt.

```
# opkg install eapol-test-openssl
```

Create configuration file for `eapol_test` with the settings for PEAP-MSCHAPv2. You can copy the following text and save it in `peap-mschapv2.conf` file in your home directory on your OpenWrt device. You can find this and other configuration files for several authentication methods here (http://deployingradius.com/scripts/eapol_test/#Testing).

```
#
# eapol_test -c peap-mschapv2.conf -a 127.0.0.1 -s testing123
#
network={
    ssid="example"
    key_mgmt=WPA-EAP
    eap=PEAP
    identity="bob"
    anonymous_identity="anonymous" # This text will be visible as a login outside the encrypted TLS channel. Not actually used for authentication.
    password="hello"
    phase2="auth=MSCHAPV2"

    #
    # Uncomment the following to perform server certificate validation.
#    ca_cert="/etc/freeradius3/certs/ca.pem"
}
```

Check that FreeRADIUS is running in debugging mode, then in another terminal session to your OpenWrt device run the following command.

```
eapol_test -c peap-mschapv2.conf -a 127.0.0.1 -s testing123
```

If all goes well, you will see a lot of text in the terminal window where you can find the `Access-Accept` message and something like the following at the end:

```
CTRL-EVENT-EAP-SUCCESS EAP authentication completed successfully
WPA: EAPOL processing complete
eapol_sm_cb: result=1
No EAP-Key-Name received from server
MPPE keys OK: 1 mismatch: 0
SUCCESS
```

At the same time, you will see text flowing through in the terminal where `radiusd -X` is running. At the end `radiusd` will show something like:

```
Sent Access-Accept Id 10 from 127.0.0.1:1812 to 127.0.0.1:51046 length 0
  MS-MPPE-Recv-Key = 0x54398b461b47f974834e81d9f9b320fd6f9a0e7da32c249e609fe7f05a37673d
  MS-MPPE-Send-Key = 0x5b7dc906a59b6ee737a99b86294b4fde5ee13ed55998ef09abac53061b0d74ca
```

The MS-MPPE-Recv-Key (RFC2548 section 2.4.3 (<https://tools.ietf.org/html/rfc2548>)) contains the Pairwise Master Key (PMK) destined to the Authenticator (access point), encrypted with the weak³⁾ MPPE (<https://tools.ietf.org/html/rfc3078>) Protocol, using the shared secret between the Authenticator and Authentication Server as key. The Supplicant derives the same PMK from Master Key, as described in Key Management section of 8021X HOWTO (https://fdp.org/HOWTO/html_single/8021X-HOWTO/#Key). PMK then is used to derive keys for WPA.

EAP-MD5-Challenge, EAP-GTC, EAP-OTP methods can not be used (<https://tools.ietf.org/html/rfc4017>) for wireless authentication environment because they do not support any of the mandatory requirements, including key derivation and mutual authentication.

If the NAS and FreeRADIUS server are not on the same host, it is recommended to move the RADIUS traffic into a separate technological VLAN() or use RadSec (or RADIUS over TLS) (<https://tools.ietf.org/html/rfc6614>) where possible, since MD5 encryption using the secret cannot be considered secure.

Configuring FreeRADIUS

Let's modify the configuration to achieve the following goals:

- configure FreeRADIUS to work as Authentication Server for WPA2 Enterprise with PEAP/MS-CHAPv2 authentication method
- store usernames and passwords in a text file
- disable insecure authentication protocols
- disable some unused functionality from the default configuration
- display a list of currently authorized users
- use examples from FreeRADIUS distribution and documentation as much as possible

All subsequent actions, unless otherwise noted, are performed in a terminal session on your OpenWrt device in `/etc/freeradius3/` directory.

Generating certificates

Digital certificates are required to do TLS()-based EAP methods, such as EAP-TLS(), PEAP, and EAP-TTLS.

You can find **test** certificates `ca.pem` (a self-signed certificate authority, i.e. root CA) and `server.pem` (a server certificate combined with an encrypted private key and signed by the CA) in the `certs/` directory immediately after installing the `freeradius3-democerts` OpenWrt package. It is suggested that new installations use the test certificates for initial tests (as many other users have exactly the same certificates and private keys), and then create real certificates to use for normal user authentication. See the instructions below for how to create the various certificates. The old test certificates can be deleted. You may continue to use `dh` parameters file from the package.

Preparation

At the moment, the FreeRADIUS distribution for OpenWrt does not contain a convenient toolkit for generating certificates and keys. Such tools are present in full-featured FreeRADIUS distributions, it is easier to use them. The description can be found in the file `raddb/certs/README` (<https://github.com/FreeRADIUS/freeradius-server/blob/v3.0.x/raddb/certs/README>) and here (<http://deployingradius.com/documents/configuration/certificates.html>).

You may also view `openssl` commands being executed in the `raddb/certs/Makefile` (<https://github.com/FreeRADIUS/freeradius-server/blob/v3.0.x/raddb/certs/Makefile>) and run them on an OpenWrt host, but this has not been tested. You will need to install the `openssl-util` OpenWrt package. See also:

- `raddb/certs/bootstrap` (<https://github.com/FreeRADIUS/freeradius-server/blob/v3.0.x/raddb/certs/bootstrap>)
- `raddb/certs/xpextensions` (<https://github.com/FreeRADIUS/freeradius-server/blob/v3.0.x/raddb/certs/xpextensions>)

Further steps are described using freeradius 3.0.16 on Ubuntu 18.04.

Open a terminal window, become a `root` , install FreeRADIUS, then go to the `certs` directory.

```
$ sudo -i
# apt update && apt install freeradius
# cd /etc/freeradius/3.0/certs
```

Notes:

- In Ubuntu `raddb` is `/etc/freeradius/3.0` and `radiusd` binary is `freeradius` .
- You may safely stop and disable the newly installed FreeRADIUS. The daemon is not required for subsequent actions.

Make a root certificate

Open file `ca.cnf` in your favorite text editor.

```
# vim ca.cnf
```

- Edit the `default_days` field in `[ CA_default ]` section. Set it to some high value (for example, `1825` for 5 years) if you don't want to re-create your certificates too often.
- Edit the `input_password` and `output_password` fields in `[ req ]` section to be the password for the CA private key. For example, `MySecureCAPassword` .

- Edit the `[certificate_authority]` section to correct the values for your country, state, etc. `countryName` should be a two-letter country code (https://en.wikipedia.org/wiki/ISO_3166-1_alpha-2), other fields may contain arbitrary information, at your option.

The following step creates:

- `ca.key` - the encrypted CA private key. This file is only required for signing new certificates, not for FreeRADIUS to work.
- `ca.pem` - CA public certificate in PEM format.
- `ca.der` - CA public certificate in DER format. DER can be imported into Windows.

```
# make ca.der
```

Make a server certificate

Open file `server.cnf` in your favorite text editor.

```
# vim server.cnf
```

- Edit the `default_days` field in `[ CA_default ]` section. Set it to some high value (for example, `1825` for 5 years) if you don't want to re-create your certificates too often.
- Edit the `input_password` and `output_password` fields in `[ req ]` section to be the password for the server private key. For example, `MySecureServerPassword`. This password must then be specified in the FreeRADIUS configuration on your OpenWrt device (see chapter below).
- Edit the `[server]` section to correct the values for your country, state, etc. Be sure that the `commonName` field here is different from the `commonName` for the CA certificate.

The following step creates:

- `server.key` - the encrypted server private key.
- `server.crt` - signed by the CA public certificate for your server.
- `server.pem` - concatenated `server.key` and `server.crt`.
- `server.p12` - combined `server.key` and `server.crt` in PKCS#12 format.
- `server.csr` - server certificate signing request. Intermediate file that was used to sign the certificate.

```
# make server.pem
```

Generate Diffie-Hellman parameters

Generate new Diffie-Hellman parameters (if you want):

```
# make dh
```

Copy files to OpenWrt device

Copy the following files to your OpenWrt device to `/etc/freeradius3/certs` directory:

- `ca.pem`
- `server.pem`
- `dh` (if you want)

Now go back to your OpenWrt device.

Adjust config

Check file permissions

Check that the files with private keys are inaccessible to the others. If they are world-accessible, adjust the permissions.

```
# chmod o= certs/server.pem
```

Set password for the server private key in EAP configurations file

Edit the `private_key_password` field in the `mods-enabled/eap` file. Set in it the password for the server private key, which was specified above in the chapter Make a server certificate in the `[req]` section in the `output_password` field.

```
#my: file: mods-enabled/eap
#my: section: eap {
#my: sub-section: tls-config tls-common {
#my: update 'private_key_password = whatever' with real password
#
#       private_key_password = whatever
#       private_key_password = MySecurePassword
```

Bind to the correct IP

Edit file `sites-enabled/default`.

Listen for authentication packets only on the desired IP() address. Set `ipaddr` to `127.0.0.1` if your Access Point is running the FreeRADIUS server.

```
#my: file: sites-enabled/default
#my: section: listen {
#my: listen type = auth
#my: change 'ipaddr = *' to myaddress
#
#       ipaddr = *
#       ipaddr = 192.168.1.1
```

The same for accounting packets.

```
#my: file: sites-enabled/default
#my: section: listen {
#my: listen type = acct
#my: change 'ipaddr = *' to myaddress
#       ipaddr = *
#       ipaddr = 192.168.1.1
```

IPv6() versions of the above. Comment out the entire sections, if you are not using IPv6().

```
#my: file: sites-enabled/default
#my: section: listen {
#my: listen type = auth
#my: comment out the entire section
#listen {
#       type = auth
#       ipv6addr = ::
#       ...
#}
#
#my: section: listen {
#my: listen type = acct
#my: comment out the entire section
#listen {
#       ipv6addr = ::
#       port = 0
#       type = acct
#       ...
#}
```

Disable insecure authentication protocols

Now let's disable insecure authorization protocols on the server side. Although this will not prevent Supplicants from starting authentication using an insecure protocol, during which their credentials can be intercepted by an attacker. Note that any authorization protocol can be used securely inside an encrypted PEAP or EAP-TTLS tunnel.

Disable non-EAP protocols

Edit the file `sites-enabled/default`.

```
#my: file: sites-enabled/default
#my: section: authorize {
#my: comment 'chap'
#    chap
#
#my: comment 'mschap'
#    mschap
#
#my: comment 'digest'
#    digest
#
#my: comment 'pap'
#    pap
```

Disable insecure EAP protocols

Edit the file `mods-enabled/eap` .

```
#my: file: mods-enabled/eap
#my: section: eap {
#my: comment out 'md5' section
#    md5 {
#    }
#
#my: comment out 'leap' section
#    leap {
#    }
#
#my: comment out 'gtc' section
#    gtc {
#    ...
#    }
```

Set PEAP as default EAP type

Edit the file `mods-enabled/eap` .

```
#my: file: mods-enabled/eap
#my: section: eap {
#my: change 'default_eap_type = md5' to peap
#    default_eap_type = md5
#    default_eap_type = peap
```

Disable EAP-MSCHAPv2 outside of EAP-TTLS and PEAP

Exclude insecure EAP-MSCHAPv2 from the methods available outside the EAP-TTLS or PEAP tunnel. `mods-enabled/eap` is referenced by `sites-enabled/default` and `sites-enabled/inner-tunnel`. So comment out the `mschapv2` configuration from `mods-enabled/eap`. To continue using EAP-MSCHAPv2 in `sites-enabled/inner-tunnel`, use an instance (https://networkradius.com/doc/current/raddb/syntax/config_names.html) of the `eap` module with a different configuration and with an instance-name `inner-eap`.

While the `inner-eap` module is not included in FreeRADIUS distribution in OpenWrt, copy its contents from source (<https://github.com/FreeRADIUS/freeradius-server/blob/3.0.x/raddb/mods-available/inner-eap>) or from the minimal configuration below:

```
eap inner-eap {
    default_eap_type = mschapv2
    timer_expire = 60
    max_sessions = ${max_requests}
    mschapv2 {
    }
}
```

Save this code in file `mods-available/inner-eap`. Set file permissions:

```
# chmod u=rw,g=r,o= /etc/freeradius3/mods-available/inner-eap
```

Then create a symbolic link in `mods-enabled/` directory:

```
# cd /etc/freeradius3/mods-enabled
# ln -s ../mods-available/inner-eap
# cd /etc/freeradius3/
```

Consider appending the line `/etc/freeradius3/mods-available/inner-eap` to the `/etc/sysupgrade.conf` file to back up this mod with the standard `sysupgrade` utility.

Add `inner-eap` section and comment out `eap` section in the file `sites-enabled/inner-tunnel`.

```
#my: file: sites-enabled/inner-tunnel
#my: section: authorize {
#my: add 'inner-eap' section; comment out 'eap' section
    inner-eap {
        ok = return
    }
#    eap {
#        ok = return
#    }
#
#my: section: authenticate {
#my: add 'inner-eap' section; comment out 'eap' section
    inner-eap
#    eap
```

Now you can comment out `mschapv2` section in the file `mods-enabled/eap` .

```
#my: file: mods-enabled/eap
#my: section: eap {
#my: comment out 'mschapv2' section; eap-mschapv2 is now in mods-enabled/inner-eap
#    mschapv2 {
#        ...
#    }
```

Disable some unused functionality

This will help free up some RAM.

Disable SQL module completely

Although the `sql` module is configured using the `-sql` construct, i.e. conditionally loaded (<https://github.com/FreeRADIUS/freeradius-server/blob/V3.0.x/raddb/mods-available/README.txt>) only if it is configured, let's comment it out.

Edit the file `sites-enabled/default` .

```
#my: file: sites-enabled/default
#my: section: authorize {
#my: comment out '-sql'
#       -sql
#
#my: section: accounting {
#my: comment out '-sql'
#       -sql
#
#my: section: post-auth {
#my: comment out '-sql'
#       -sql
#
#my: section: post-auth {
#my: sub-section: Post-Auth-Type REJECT {
#my: comment out '-sql'
#       -sql
```

And in the same way the file `sites-enabled/inner-tunnel`.

```
#my: file: sites-enabled/inner-tunnel
#my: section: authorize {
#my: comment out '-sql'
#       -sql
#
#my: section: post-auth {
#my: comment out '-sql'
#       -sql
#
#my: section: post-auth {
#my: sub-section: Post-Auth-Type REJECT {
#my: comment out '-sql'
#       -sql
```

Disable LDAP module completely

Although the `ldap` module is configured using the `-ldap` construct, i.e. ~~conditionally loaded~~ (<https://github.com/FreeRADIUS/freeradius-server/blob/v3.0.x/raddb/mods-available/README.txt>) only if it is configured, let's comment it out.

Edit the file `sites-enabled/default` .

```
#my: file: sites-enabled/default
#my: section: authorize {
#my: comment out '-ldap'
#      -ldap
```

And in the same way the file `sites-enabled/inner-tunnel` .

```
#my: file: sites-enabled/inner-tunnel
#my: section: authorize {
#my: comment out '-ldap'
#      -ldap
```

Do not create a 'detail'ed log of the packets

Comment out the `detail` module in the `accounting` section in file `sites-enabled/default` . Request packets will not be logged in `/var/db/radacct/` . This will save several hundred KB in RAM.

```
#my: file: sites-enabled/default
#my: section: accounting {
#my: comment out 'detail' to save some space in tmpfs
#      detail
```

Turn proxying off

Our system will not proxy requests to other servers, then turn proxying off in the file `radiusd.conf` . This will save [save \(https://networkradius.com/doc/current/raddd/radiusd.html\)](https://networkradius.com/doc/current/raddd/radiusd.html) a small amount of resources on the server. To disable proxying, change `proxy_requests` value from `yes` to `no` , and comment out the `$INCLUDE proxy.conf` line.

```
#my: file: radiusd.conf
#my: change 'proxy_requests = yes' to no; comment out '$INCLUDE proxy.conf' as not used
proxy_requests = no
#$INCLUDE proxy.conf
```

Add production NAS's and users

Add NAS's

Edit the `clients.conf` file.

1. Comment out the test client section if you added it in Set up FreeRADIUS for initial testing.
2. Add each of your Access Points to the end:

```
# Access Point 1
client ap1 {
    ipaddr = 192.168.1.11    # change it according to the IP address of your AP1
    secret = SecretForAP1    # you must configure the same secret on the Access Point 1.
}
# Access Point 2
client ap2 {
    ipaddr = 192.168.1.12
    secret = SecretForAP2
}
# Access Point N
# ...
```

Add users

Edit the file `mods-config/files/authorize` .

1. Comment out the test user `bob` , that was enabled earlier in Set up FreeRADIUS for initial testing.
2. Add *logins* and *passwords* for all your users to the beginning of the file.

```
# User 1
user1    Cleartext-Password := "password1"
# User 2
user2    Cleartext-Password := "password2"
# User N
# ...
```

Settings for displaying a list of authorized users

Configure FreeRADIUS to maintain a file about logged in users. Edit the file `sites-enabled/default` . Accounting must not be disabled.

```
#my: file: sites-enabled/default
#my: section: accounting {
#my: uncomment 'radutmp'
    radutmp
```

Inform the NAS about the real user name. The NAS will return this name to the server in accounting packages. Edit the file `sites-enabled/inner-tunnel`.

```
#my: file: sites-enabled/inner-tunnel
#my: section: post-auth {
#my: uncomment 'update outer.session-state' section
    update outer.session-state {
        User-Name := &User-Name
    }
```

Now you can run the command `radwho` from `freeradius3-utils` package and see the list of logged in users. Unfortunately, the documentation says that due to packet losses in the network, the data here may be incorrect.

OpenWrt does not provide the `last` command. Therefore, the `radlast` command is not available for displaying a list of the last authorized users. However, you can copy the `/var/log/radwtmp` file to another machine where the `last` command is available. There you can view the contents of the `radwtmp` file:

```
last -f /path/to/downloaded/radwtmp
```

Run FreeRADIUS

When the configuration is complete and successfully tested in debugging mode, start the daemon in standard way.

```
/etc/init.d/radiusd start
```

Note:

- You must stop the `radiusd` server and start it again after you modified the configuration files.

Configure Wireless Access Point

802.1x authentication is only available with full `wpad` package, `wpad-mini` does not have the required modules in it. So install `wpad` and remove `wpad-mini` with

```
# opkg remove wpad-mini ; opkg install wpad
```

If there is not enough space on the flash memory, try to build the modified firmware with the necessary packages using the image builder.

Configuration of the `wifi-iface` section in file `/etc/config/wireless` on Wireless Access Point should be similar to the following:

```
config wifi-iface 'default_radio0'
    option device 'radio0'
    option network 'lan'
    option mode 'ap'
    option ssid 'OpenWrt'
    option encryption 'wpa2'                # 'wpa2' for WPA2 Enterprise instead of 'psk2' for WPA2 Personal (PSK).
    option server '192.168.1.1'             # RADIUS server IP. Set to 127.0.0.1 if your Access Point runs the FreeRADIUS server itself.
In older OpenWrt releases, this option may be named 'auth_server'.
    option key 'SecretForAP1'               # the 'key' must match the 'secret' for the corresponding client from the file '/etc/freeradi
us3/clients.conf'. In older OpenWrt releases, this option may be named 'auth_secret'
    option acct_server '192.168.1.1'        # Configure this option to enable accounting.
    option acct_secret 'SecretForAP1'
```

Configure user devices

When configuring a user device (Supplicant), specify the following values:

- Security: WPA & WPA2 Enterprise
- Authentication: Protected EAP (PEAP)
- Anonymous identity: anonymous
- CA certificate: (import and select root certificate ca.pem or ca.der)
- Inner authentication: MSCHAPv2
- Username: user1
- Password: password1

Note:

- WDS config does not work, if you want to extend Wireless coverage with multiple APs, you will need Wifi Extender through relayd config instead. The discussion for this can be found on OpenWrt Forum here (<https://forum.openwrt.org/t/wds-client-with-802-1x/27994>).

Conclusion

In case you have any problems configuring or installing, create a topic at OpenWrt Forum and mention username **ahmar16**.

Useful Resources

- Online FreeRADIUS documentation <https://networkradius.com/doc/current/> (<https://networkradius.com/doc/current/>)

- Collection of official FreeRADIUS PDFs: Technical Guide, PAP Authentication, EAP Authentication, etc. <https://networkradius.com/technology/freeradius/> (<https://networkradius.com/technology/freeradius/>)
- Number of FreeRADIUS “Howto” documents <http://deployingradius.com/> (<http://deployingradius.com/>)
- FreeRADIUS Documentation <https://freeradius.org/documentation/> (<https://freeradius.org/documentation/>)
- FreeRADIUS source code on GitHub <https://github.com/FreeRADIUS/freeradius-server> (<https://github.com/FreeRADIUS/freeradius-server>)
- 802.1X Port-Based Authentication HOWTO https://tldp.org/HOWTO/html_single/8021X-HOWTO/ (https://tldp.org/HOWTO/html_single/8021X-HOWTO/)

- 1) <https://www.kb.cert.org/vuls/id/836068> (<https://www.kb.cert.org/vuls/id/836068>)
- 2) <https://www.schneier.com/wp-content/uploads/2015/12/paper-pptpv2.pdf> (<https://www.schneier.com/wp-content/uploads/2015/12/paper-pptpv2.pdf>)
- 3) <https://www.sans.org/security-resources/malwarefaq/pptp-vpn> (<https://www.sans.org/security-resources/malwarefaq/pptp-vpn>)

This website uses cookies. By using the website, you agree with storing cookies on your computer. Also you acknowledge that you have read and understand our Privacy Policy. If you do not agree leave the website. More information about cookies

 Last modified: 2021/04/18 16:07 by tmomas

Self-registration in the wiki has been disabled.
If you want to contribute to the OpenWrt wiki, please post **HERE** (<https://forum.openwrt.org/t/applying-for-openwrt-wiki-account/101671>) in the forum or ask on **IRC** () for access.



Except where otherwise noted, content on this wiki is licensed under the following license:
CC Attribution-Share Alike 4.0 International